

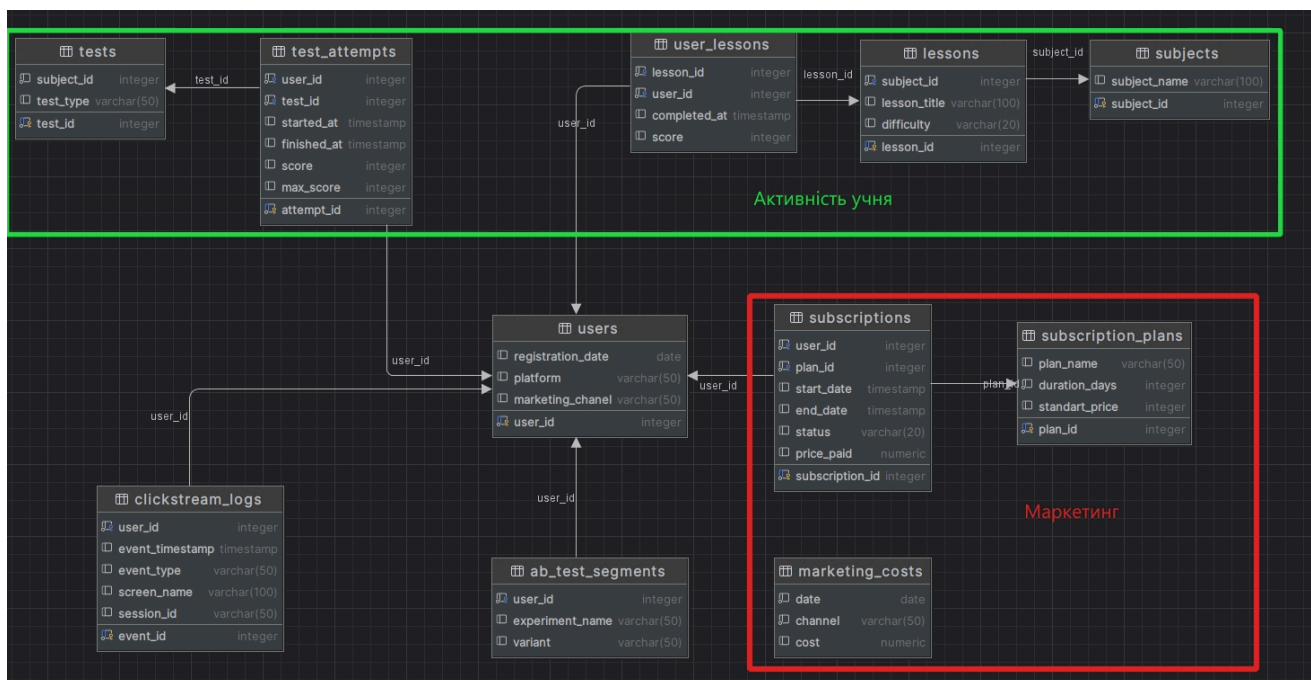
Ed-tech proj

T3: Аналіз продуктових та маркетингових даних

1. Вхідні дані та структура датасету

Датасет містить лог-дані активності користувачів мобільного додатка:

- **Users / Registrations:** `user_id`, `registration_date`, `platform` (iOS/Android), `marketing_channel`.
- **Clickstream Logs:** `user_id`, `session_id`, `event_timestamp`, `screen_name`.
- **Marketing Costs:** `date`, `channel`, `acquisition_cost` (загальні щоденні витрати на канал).
- **Lessons & Tests:** `user_id`, `lesson_id`, `subject`, `started_at`, `completed_at`, `score`.
- **A/B Test Data:** `user_id`, `group` (A/B), `platform`.



2. Основні завдання

Task 1: Ad-Hoc SQL Аналіз

- Розрахувати середній час складання тестів користувачами.
- Проранжувати найскладніші уроки у розрізі предметів.
- Оцінити утримання користувачів (Retention) на основі дат активності.

Task 2: Аудит логів та воронка конверсії

- Перевірити клікстрім-логи на наявність подвоєння даних та фронтенд-помилки.
- Побудувати покрокову воронку проходження Onboarding.

Task 3: Оцінка А/Б Тесту

- Розрахувати статистичну значущість результатів експерименту.
- Проаналізувати якість сплітування трафіку (перевірити наявність SRM та сегментних викривлень).
- Сформулювати фінальну рекомендацію щодо викочування фічі.

Task 4: Розрахунок Unit-економіки

- Обчислити ключові метрики продукту: **DAU, MAU, ARPU, ARPPU, LTV, CAC, ROMI**.
- Вірно згрупувати маркетингові витрати з доходами користувачів.

Task 5: Дашборд у Power BI

- Розробити дашборд для Product Owner з інтерактивними фільтрами (платформа, канал, час).
- Реалізувати розрахунок DAX-метрика з динамічним зрізом показників.

3. Виконання завдання

Створимо для початку базу даних для зручної роботи з sql (приклад таблиці та створення ключа)

```
CREATE TABLE users (
  user_id INT NOT NULL ,
  registration_date DATE NULL,
  platform VARCHAR (50) NULL,
  marketing_chanel VARCHAR (50) NULL );
```

```
ALTER TABLE ab_test_segments ADD CONSTRAINT FK_ab_test_segments
FOREIGN KEY (user_id)
REFERENCES users(user_id)
ON DELETE CASCADE ;
```

Скористаємося ORM для відправки даних на базу (приклад створення таблиці та відправки)

```
load_dotenv()

DBCON = os.getenv('DBCON')

engine = create_engine(f"postgresql://postgres:{DBCON}@ed_tech_proj")
```

```

SessionFactory = sessionmaker(bind=engine)  Base = declarative_base()

class Users(Base):
    __tablename__ = 'users'

    user_id = Column(Integer, primary_key=True)
    registration_date = Column(Date)
    platform = Column(String(50))
    marketing_chanel = Column(String(50))

class Subscriptions(Base):
    __tablename__ = 'subscriptions'

    subscription_id = Column(Integer, primary_key=True)
    user_id = Column(Integer, nullable=False)
    plan_id = Column(Integer, nullable=False)
    start_date = Column(Date)
    end_date = Column(Date)
    status = Column(String(20))
    price_paid = Column(Numeric)
    users.rename(columns={'marketing_channel': 'marketing_chanel'}).to_sql(
        name='users',
        con=engine,
        if_exists='append',
        index=False  )

```

Перша частина завдань (Ad-Hoc)

Виконаємо перше завдання на пошук ретеншн. Так як даних з логів недостатньо, будемо відслідковувати активність учнів через уроки та здачу тестів

```

WITH user_activity AS ( -- беремо всі активності користувача та їх дати
    SELECT A.user_id, A.registration_date , B.completed_at
    FROM users A INNER JOIN user_lessons B
    ON A.user_id = B.user_id
    UNION ALL
    SELECT A.user_id , A.registration_date, B.finished_at
    FROM users A INNER JOIN test_attempts B
    ON A.user_id = B.user_id),

user_stat AS ( -- індикатор, чи активний користувач на n-ий день
    SELECT DISTINCT ON (user_id) *, CASE WHEN latest_activity >= 1 THEN 1
    ELSE 0 END day1 , CASE WHEN latest_activity > 6 THEN 1 ELSE 0 END day7, CASE
    WHEN latest_activity > 29 THEN 1 ELSE 0 END day30
    FROM ( -- шукаємо найпізнішу активність
        SELECT *, LAST_VALUE(completed_at) over (PARTITION BY user_id ORDER
        BY completed_at rows between unbounded preceding and UNBOUNDED

```

```

FOLLOWING)::date - registration_date AS latest_activity
FROM user_activity
WHERE registration_date BETWEEN '2026-02-01' AND '2026-03-01')Q --
метрика за лютий
ORDER BY user_id, latest_activity DESC)

SELECT CAST(SUM(day1) AS DECIMAL)/COUNT(*) AS retention_rate_1day,
CAST(SUM(day7) AS DECIMAL)/COUNT(*) AS retention_rate_7day, CAST(SUM(day30)
AS DECIMAL)/COUNT(*) AS retention_rate_30day -- співвідношення FROM
user_stat;

```

retention_rate_1day	retention_rate_7day	retention_rate_30day
0.99456521739130434783	0.9375	0

Як бачимо в основном користуються продуктом проятгом перших 7 днів, до 30 майже ніхто не доходить

retention_rate_1day	retention_rate_7day	retention_rate_30day
0.99152310637134263057	0.94667760459392945037	0

Ситуація для січня аналогічна.

Знайдемо найскладніші уроки для кожного прдемету, одночасно перевіривши декілька гіпотез.

```

WITH merged_table as (
    SELECT b.lesson_id, B.user_id, B.score, C.subject_id
    FROM user_lessons B INNER JOIN lessons C
    ON B.lesson_id = C.lesson_id),

-- SELECT lesson_id, COUNT(lesson_id) -- перевіримо кількість пройдених
уроків, щоб не прийняти непопулярні за складні -- FROM merged_table --
GROUP BY lesson_id

-- SELECT COUNT(*), COUNT(DISTINCT (user_id, lesson_id)) -- переконаємось,
що кожен учень проходить урок один раз -- FROM merged_table

lesson_rate as (
    SELECT subject_id, lesson_id, AVG(score) as score
    FROM merged_table
    GROUP BY subject_id, lesson_id),

lesson_rank as (
    SELECT *, ROW_NUMBER() over (PARTITION BY subject_id ORDER BY score ASC)

```

```
AS rnk
    FROM lesson_rate
)
```

```
SELECT * FROM lesson_rank WHERE rnk <= 3 ORDER BY subject_id, rnk;
```

subject_id	lesson_id	score	rnk
1	3	79.6627974463145676	1
1	5	79.8119103773584906	2
1	1	79.8660229330114665	3
2	8	79.8668280871670702	1
2	7	79.9851895734597156	2
2	10	79.9891435464414958	3
3	12	79.5858024691358025	1
3	11	79.9777242624924744	2
3	14	80.0401197604790419	3

Як кінцевий запит - знайдемо середній час здачі та перевіримо на аномалії (чи хтось абюзить баг).

```
SELECT * -- один учень - одна успішна спроба FROM (
    SELECT user_id, COUNT(user_id) AS cnt
    FROM test_attempts A INNER JOIN tests B
    ON A.test_id = B.test_id
    WHERE score > 140
    GROUP BY user_id)q WHERE cnt > 1;
```

```
SELECT AVG(finished_at - started_at) FROM test_attempts A INNER JOIN tests
B ON A.test_id = B.test_id WHERE score > 140;
```

```
SELECT MAX(finished_at - started_at) -- підтвердимо припущення, що макс 2
години FROM test_attempts A INNER JOIN tests B ON A.test_id = B.test_id
WHERE score > 140;
```

```
SELECT MIN(finished_at - started_at) -- пошук аномалій FROM test_attempts A
INNER JOIN tests B ON A.test_id = B.test_id WHERE score > 140
```

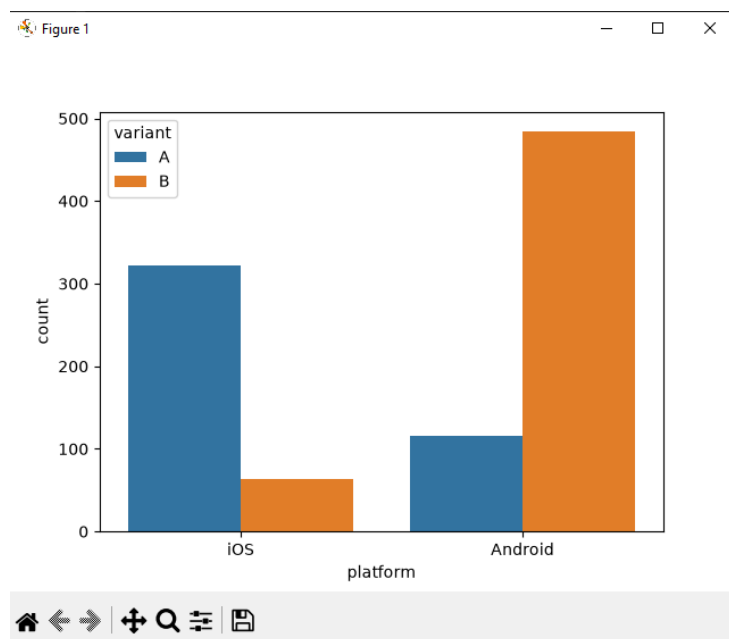
0 years 0 mons 0 days 1 hours 28 mins 18.142549 secs -avg

0 years 0 mons 0 days 1 hours 59 mins 0.0 secs - max

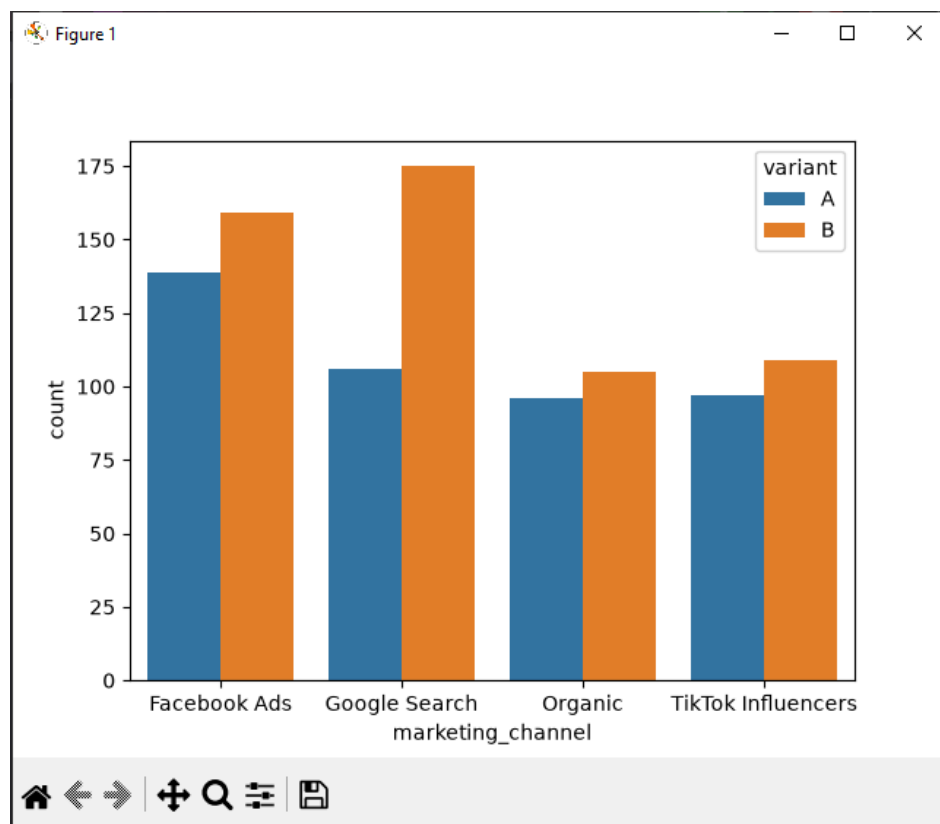
0 years 0 mons 0 days 1 hours 0 mins 0.0 secs - min

АБ-тестування

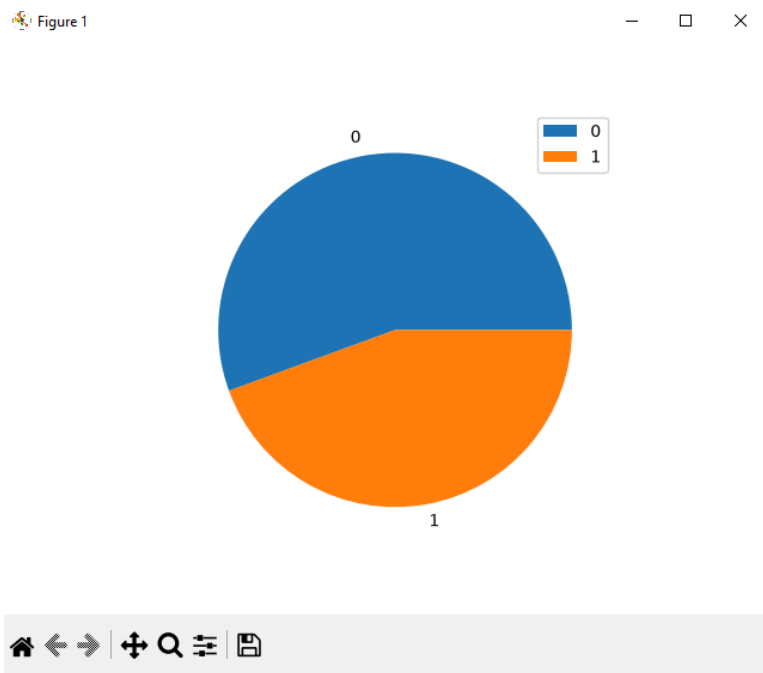
Проведемо аб тест попередньо дослідивши вибірки



Як бачимо для групи А домінує айос, в той час як для Б андроїд. Це є поганою практикою для аб тесту, так як платформа може безпосередньо свідчити про поведінку користувача, що змушує переглянути систему розподілу (рандомізацію).



Для залучення все стабільніше, проте, якщо виявиться, що google search є більш ефективним, варіант Б може отримати несправедливий баф.



Розподіл між групами майже рівний, проте не дотриманий золотий стандарт 50 на 50. Далі йде основна частина експерименту.

```
users_ab_test_subscription = users_ab_test.merge(subscriptions, on =
'user_id', how = 'left') print(users_ab_test_subscription)
users_ab_test_subscription_a =
users_ab_test_subscription[users_ab_test_subscription['variant'] == 'A']
users_ab_test_subscription_b =
users_ab_test_subscription[users_ab_test_subscription['variant'] == 'B']

cra = users_ab_test_subscription_a['subscription_id'].count() /
users_ab_test_subscription_a['user_id'].count() crb =
users_ab_test_subscription_b['subscription_id'].count() /
users_ab_test_subscription_b['user_id'].count()

print(f'cra : {cra}') print(f'crb : {crb}')

pa = cra pb = crb ppool =
(users_ab_test_subscription_a['subscription_id'].count() +
users_ab_test_subscription_b['subscription_id'].count())/
(users_ab_test_subscription_a['user_id'].count() +
users_ab_test_subscription_b['user_id'].count()) SEppool = np.sqrt(ppool*
(1-ppool)*(1/users_ab_test_subscription_a['user_id'].count() +
1/users_ab_test_subscription_b['user_id'].count())) z = (cra - crb)/SEppool
if z > 1.96 :
print(f'feature a statistically significant, z-score = {z}') if z <= 1.96 :
    print(f'bad feature a, z-score = {z}')
```

cra : 0.0684931506849315

crb : 0.05656934306569343

bad feature a, z-score = 0.772226975783667

Як бачимо жоден з варіантів не виправдовую себе. Не дивлячись на велику різницю у конверсії даних досить мало для того, щоб робити якісь висновки. Для того щоб не втрачати потенційну золоту жилу - найкращий варіант продовжити тест для зменшення дисперсії та звуження хвостів довірчих інтервалів.

Як прагматичні аналітики додатково дослідимо сегменти.

```
# різні сегменти для a  cra_by_cohorts =
users_ab_test_subscription_a.groupby(['platform', 'marketing_channel'])
[['user_id', 'subscription_id']].count().reset_index()  cra_by_cohorts['cr']
= cra_by_cohorts['subscription_id'] / cra_by_cohorts['user_id']
print(cra_by_cohorts)

# різні сегменти для b  crb_by_cohorts =
users_ab_test_subscription_b.groupby(['platform', 'marketing_channel'])
[['user_id', 'subscription_id']].count().reset_index()  crb_by_cohorts['cr']
= crb_by_cohorts['subscription_id'] / crb_by_cohorts['user_id']
print(crb_by_cohorts)

# різні сегменти для загальної сукупності  cr_by_cohorts =
users_ab_test_subscription.groupby(['platform', 'marketing_channel'])
[['user_id', 'subscription_id']].count().reset_index()  cr_by_cohorts['cr'] =
cr_by_cohorts['subscription_id'] / cr_by_cohorts['user_id']
print(cr_by_cohorts)

# cr по каналах  cr_by_marketing_channel =
users_ab_test_subscription.groupby('marketing_channel')
[['user_id', 'subscription_id']].count().reset_index()
cr_by_marketing_channel['cr'] = cr_by_marketing_channel['subscription_id'] /
cr_by_marketing_channel['user_id']  print(cr_by_marketing_channel)

# cr по платформах  cr_by_platform =
users_ab_test_subscription.groupby('platform')
[['user_id', 'subscription_id']].count().reset_index()  cr_by_platform['cr']
= cr_by_platform['subscription_id'] / cr_by_platform['user_id']
print(cr_by_platform)

Na = users_ab_test_subscription_a['user_id'].count()  Nb =
users_ab_test_subscription_b['user_id'].count()
```

	platform	marketing_channel	user_id	subscription_id	cr
0	Android	Facebook Ads	43	2	0.046512
1	Android	Google Search	27	0	0.000000
2	Android	Organic	21	2	0.095238
3	Android	TikTok Influencers	25	1	0.040000
4	iOS	Facebook Ads	96	7	0.072917
5	iOS	Google Search	79	6	0.075949

6 iOS Organic 75 8 0.106667
 7 iOS TikTok Influencers 72 4 0.055556
 platform marketing_channel user_id subscription_id cr
 0 Android Facebook Ads 136 12 0.088235
 1 Android Google Search 155 7 0.045161
 2 Android Organic 95 5 0.052632
 3 Android TikTok Influencers 98 0 0.000000
 4 iOS Facebook Ads 23 3 0.130435
 5 iOS Google Search 20 4 0.200000
 6 iOS Organic 10 0 0.000000
 7 iOS TikTok Influencers 11 0 0.000000
 platform marketing_channel user_id subscription_id cr
 0 Android Facebook Ads 179 14 0.078212
 1 Android Google Search 182 7 0.038462
 2 Android Organic 116 7 0.060345
 3 Android TikTok Influencers 123 1 0.008130
 4 iOS Facebook Ads 119 10 0.084034
 5 iOS Google Search 99 10 0.101010
 6 iOS Organic 85 8 0.094118
 7 iOS TikTok Influencers 83 4 0.048193
 marketing_channel user_id subscription_id cr
 0 Facebook Ads 298 24 0.080537
 1 Google Search 281 17 0.060498
 2 Organic 201 15 0.074627
 3 TikTok Influencers 206 5 0.024272
 platform user_id subscription_id cr
 0 Android 600 29 0.048333
 1 iOS 386 32 0.082902

Найкраща платформа - фейсбук, найкращий застосунок - айос, проте дійсно найсильнішою комбінацією виявилось гугл айос, органік айос, фейсбук айос. Через домінацію айос в варіанті А, можна пояснити чому він демонструє кращі варіанти. У висновку можна сказати, що слід звернути увагу на айфонузерів(реклама апп стор) й якщо прям хочеться пререверити гіпотезу - пофіксити рандомайзер та повернутися до екперименту знову. Спробуємо прояснити ситуацію за допомогою bootstrap

```

## bootstrap

diff_distribution = []

for i in range(10000):
    sample_a = users_ab_test_subscription_a.sample(n = Na, replace = True)
    sample_b = users_ab_test_subscription_b.sample(n=Nb, replace = True)
    cra_sample = sample_a['subscription_id'].count() /
    sample_a['user_id'].count()
  
```

```

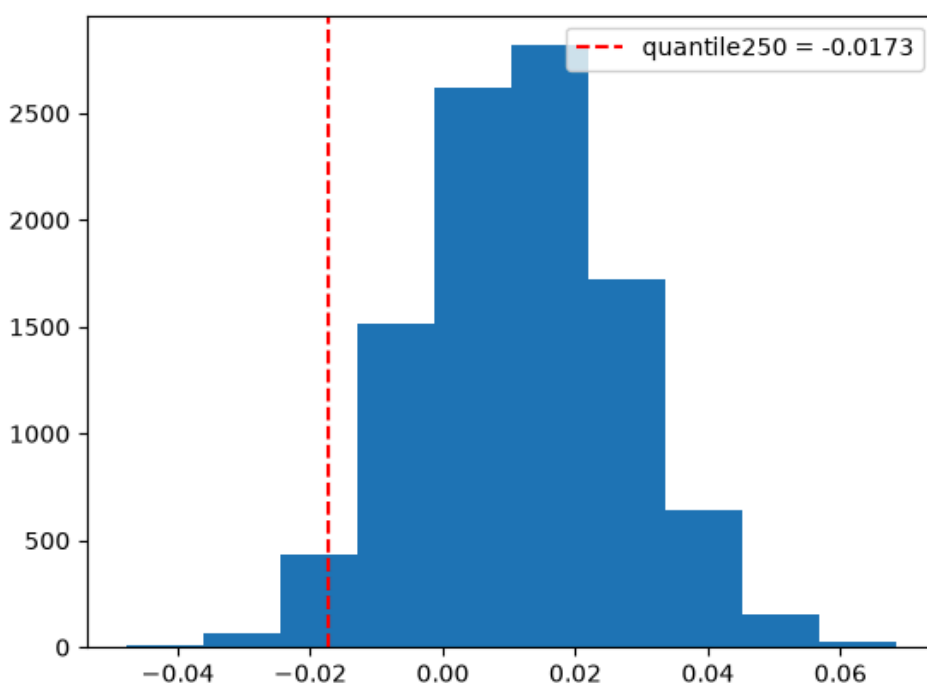
    crb_sample = sample_b['subscription_id'].count() /
sample_b['user_id'].count()
    diff = cra_sample - crb_sample
    diff_distribution.append(diff)

diff_distribution.sort()  quantile250 = diff_distribution[250]  quantile9750
= diff_distribution[9750]

plt.hist(diff_distribution)  plt.axvline(quantile250, color='red',
linestyle='--', label=f'quantile250 = {quantile250:.4f}')  plt.legend()
plt.show()

print(quantile250, quantile9750)

```



Також бачимо відсутність стат значимості.

-0.01728993767289938 0.042954037929540376

Метрики для каналів залучення

Зробимо загальні висновки

```

-- SELECT max(end_date) -- FROM subscriptions -- WHERE status = 'Expired';

-- SELECT min(end_date) -- FROM subscriptions -- WHERE status = 'Active';

-- SELECT * -- FROM subscriptions -- WHERE end_date BETWEEN '2026-03-
28'::date AND '2026-03-31'::date;

-- SELECT * -- FROM subscriptions -- WHERE end_date < '2026-03-31'::date

```

```
AND status = 'Expired';
```

```
-- всі підписки активні після 03-31 - дійсні -- всі підписки не активні  
після 03-28 - протерміновані
```

Знайдемо усіх залучених

```
CREATE VIEW user_sub AS ( -- загальний вигляд, пов'язаний з підписками  
    SELECT a.user_id, registration_date, platform, marketing_chanel,  
           subscription_id, b.plan_id, standart_price, start_date, end_date,  
price_paid,  
           plan_name, duration_days, cost as acquistion_cost  
    --EXTRACT(DAY FROM end_date - start_date)  
    FROM users A LEFT JOIN subscriptions B  
    ON A.user_id = B.user_id  
    LEFT JOIN subscription_plans C  
    ON B.plan_id = C.plan_id  
    LEFT JOIN marketing_costs D  
    ON A.marketing_chanel = D.channel AND a.registration_date::date = d.date  
    -- AND d.date = (SELECT MIN(date) FROM marketing_costs WHERE  
a.registration_date::date <= date); -- якщо були б пропуски в розцінках між  
днями -- WHERE price_paid < standart_price  
    -- WHERE duration_days = EXTRACT(DAY FROM end_date - start_date));
```

--ціна оплати може бути нижчою за стандартну, але ніколи не вищою за стандартну!

```
--канали SELECT DISTINCT (users.marketing_chanel) FROM users;
```

```
--кількість користувачів SELECT marketing_chanel, COUNT(users.user_id)  
FROM users GROUP BY marketing_chanel;
```

marketing_chanel	count
Facebook Ads	1536
TikTok Influencers	980
Organic	1004
Google Search	1480

```
SELECT marketing_chanel, sum(price_paid) / COUNT (user_id), sum(price_paid)  
/ COUNT(plan_id) FROM user_sub GROUP BY marketing_chanel;
```

marketing_chanel	?column?	?column?
Facebook Ads	29.3359375	455.15151515151515
TikTok Influencers	24.836734693877551	442.54545454545455
Organic	20.398406374501992	409.6
Google Search	27.1824324324324324	423.4736842105263158

```
-- -- загальний дохід по каналах SELECT marketing_chanel, SUM(price_paid)
FROM user_sub GROUP BY marketing_chanel ;
```

```
-- -- розцінки без пропусків -- SELECT date, COUNT(date) -- FROM
marketing_costs -- GROUP BY date -- HAVING COUNT(date) <> 3
```

marketing_chanel	sum
Facebook Ads	45060
TikTok Influencers	24340
Organic	20480
Google Search	40230

Дохід не залежить від помилки з costами - лишається без змін. А ось витрати рахую по-новому.

```
-- витрачено всього на кампанію (рахуємо напряму з marketing_costs, без
дублювання на кожного користувача) SELECT channel, SUM(cost) AS spend FROM
marketing_costs GROUP BY channel ;
```

channel	spend
Facebook Ads	35602.91
Google Search	26924.64
TikTok Influencers	22060.20
Organic	- (витрат немає, органічний трафік)

```
-- вартість залучення користувача (CAC) SELECT marketing_chanel,
spend/users_acquisition AS cac FROM(
    SELECT marketing_chanel, COUNT(user_id) as users_acquisition
    FROM user_sub
    GROUP BY marketing_chanel)Q INNER JOIN
(SELECT channel, SUM(cost) as spend
```

```

FROM marketing_costs
GROUP BY channel )B ON Q.marketing_chanel = B.channel ;

-- вартість залучення преміум користувача (тільки ті, хто купив підписку)
SELECT marketing_chanel, spend/users_acquisition AS cac_premium FROM(
    SELECT marketing_chanel, COUNT(user_id) FILTER ( WHERE subscription_id
IS NOT NULL ) users_acquisition
    FROM user_sub
    GROUP BY marketing_chanel)Q INNER JOIN
    (SELECT channel, SUM(cost) as spend
    FROM marketing_costs
    GROUP BY channel )B ON Q.marketing_chanel = B.channel ;

```

marketing_chanel	cac
Google Search	18.19
TikTok Influencers	22.51
Facebook Ads	23.18
Organic	-

marketing_chanel	cac_premium
Google Search	283.42
Facebook Ads	359.63
TikTok Influencers	401.09
Organic	-

Замість шаленого CAC в тисячі доларів (як було з баганим acquisition_cost) - реалістичні цифри, порівнянні з ARPPU по каналах (303-455).

```

-- ромі, тепер від реальних витрат замість продубльованого acquisition_cost
SELECT Q.marketing_chanel, (revenue - spend) / spend as profit FROM (
    SELECT marketing_chanel, SUM(price_paid) as revenue
    FROM user_sub
    GROUP BY marketing_chanel) Q
INNER JOIN
    (SELECT channel, SUM(cost) as spend
    FROM marketing_costs
    GROUP BY channel) B ON Q.marketing_chanel = B.channel ORDER BY profit DESC

```

marketing_chanel	profit
Google Search	0.494170
Facebook Ads	0.265627

marketing_chanel	profit
TikTok Influencers	0.103344
Organic	- (немає витрат, тому не порахувати)

Як бачимо після фіксу картина протилежна попередній - усі платні канали фактично в плюсі, а не в глибокому мінусі. Найрентабельніша реклама - google search (+49%), tiktok навпаки найслабший з платних каналів (+10%). Раніше баг з costami (денна сума на канал приймалась за ціну за одного користувача і дублювалась на кожного залученого того дня) робив CAC астрономічним і топив гомі в мінус по всіх каналах - висновок про "провальну кампанію" був хибним.

Проверка на аномалии в логах

Найчастіше збої в логах представляють з себе повторну відправку одних й тих же даних. Перевіримо записи на однаковий час.

```
-- SELECT * -- FROM clickstream_logs -- WHERE event_timestamp::date =
'2026-02-15' -- ORDER BY session_id, event_timestamp ASC;

-- -- перевірка повторного часу SELECT * FROM (
    SELECT *,COUNT (event_timestamp) OVER (PARTITION BY user_id,
event_timestamp) as unique_event
    FROM clickstream_logs
)Q WHERE unique_event > 1 ORDER BY user_id, event_timestamp
```

Такі дійсно є, проте користувач міг виконати дію протягом однієї хвилини. Додамо перевірку на сторінку.

```
-- -- перевірка на повний дублікат SELECT * FROM (
    SELECT *,COUNT (event_timestamp) OVER (PARTITION BY user_id,
event_timestamp, screen_name) as unique_event
    FROM clickstream_logs
)Q WHERE unique_event > 1 ORDER BY user_id, event_timestamp
```

Нічого не повертає, що свідчить про відсутність таких випадків. Перевіримо чи трапляється така ситуація, коли в одного користувача декілька сесій (та проаналізуємо випадки поточною)

```
-- -- Перевірка на дублювання логів (кількість сесій перевищує кількість
користувачів) SELECT event_timestamp::date, COUNT(DISTINCT(session_id)) as
sessions, COUNT(DISTINCT (user_id)) as users FROM clickstream_logs GROUP
BY event_timestamp::date -- HAVING COUNT(DISTINCT(session_id))
<>COUNT(DISTINCT (user_id))
```

Таких не знайшлося. Переглянемо чи співпадають дві дії підряд.

```
WITH error_logs AS( -- ПОВТОРЮВАНІ ЛОГИ
  SELECT *
  FROM(
    SELECT *, screen_name as actual_event, LEAD(screen_name) OVER
    (PARTITION BY user_id ORDER BY session_id, event_id) as next_event
    FROM clickstream_logs)Q
  WHERE actual_event = next_event)

-- DELETE FROM error_logs -- видалення дублікатів -- WHERE event_id IN ( -
-   SELECT event_id --   FROM error_logs --   )

SELECT COUNT(DISTINCT (session_id)) -- кількість дублікатів FROM
error_logs;
```

БІНГО! Дійсно є місце багу. Проте важливо зауважити, що якщо я робив перевірку по часу, вона нічого не продемонструвала б. Послідовність дій можна простежити по event_id проте він суперечить часу (event 543 може йти раніше event 541), що є прогалиною при генерації даних. Видалимо зайві рядки (я не видаляв для збереження датасету). До того ж занепокоєння стосовно dau зайве, адже кількість користувачів не зросла в день. Про це може ввести в оману при аналізі того ж дослідження фіч.

Проведемо дослідження воронки

```
-- загальна кількість складових воронки SELECT screen_name,
COUNT(screen_name) FROM clickstream_logs GROUP BY screen_name;

SELECT * FROM (
  SELECT user_id, session_id, event_id, screen_name as first_step,
  LEAD(screen_name, 1) OVER (PARTITION BY session_id ORDER BY event_id) as
  second_step,
    LEAD(screen_name, 2) OVER (PARTITION BY session_id ORDER BY
  event_id) as third_step,
    LEAD(screen_name, 3) OVER (PARTITION BY session_id ORDER BY
  event_id) as fourth_step,
    LEAD(screen_name, 4) OVER (PARTITION BY session_id ORDER BY
  event_id) as fifth_step
  FROM clickstream_logs)Q WHERE event_id = (SELECT MIN(event_id) FROM
  clickstream_logs WHERE Q.session_id = clickstream_logs.session_id) AND
  (first_step <> 'Onboarding_1' OR second_step <> 'Onboarding_2' OR
  third_step <> 'Main_Dashboard' OR fourth_step <> 'Lesson_List' OR fifth_step
  <> 'Paywall');
```

screen_name	count
Main_Dashboard	1505

screen_name	count
Paywall	1501
Lesson_List	1504
Onboarding_2	1505
Onboarding_1	1504

Доволі дивний результат, у який тяжко віритися, перевіримо данні під капотом за допомогою віконих функцій. Більшість записів зберігають класичну послідовність (онбординг1 - онбординг2 - ...), проте є виключення (ті ж випадки за 15 число).

Візуалізація

Імпортуємо дані з бд. Найголовнішою метрикою вважаю відношення LTV/CAC проте через дуже малу вибірку (аналізувати для періода 3 місяця можна лише одну когорту). Тож поки створено костиль для обмеженого періоду. Як рахуємо? Для початку знаходимо для кожного покупця потенційну поведінку для кожного періоду часу.

```
WITH generated_date AS ( -- список дат
    SELECT generate_series('2026-01-01'::date, '2026-05-01'::date, '1
month'::interval) AS generated_date
),

user_sub_history AS ( -- історія покупок для всіх користувачів з розбивкою
по місяцях
    SELECT *
    FROM users A FULL JOIN generated_date B
    ON 1 = 1
    LEFT JOIN subscriptions C ON
    A.user_id = C.user_id AND EXTRACT(month FROM B.generated_date) =
EXTRACT(month FROM C.start_date)
    WHERE EXTRACT(month FROM A.registration_date) <= EXTRACT(month FROM
B.generated_date) AND EXTRACT(month FROM registration_date) = 1
    ORDER BY A.user_id)

SELECT generated_date, -- підрахунок кожної опції для кожного місяця
    COUNT(*) FILTER ( WHERE plan_id = 1 ) AS plan1,
    COUNT(*) FILTER ( WHERE plan_id = 2 ) AS plan2,
    COUNT(*) FILTER ( WHERE plan_id = 3 ) AS plan3,
    COUNT(*) FILTER ( WHERE plan_id IS NULL ) AS nul FROM user_sub_history
GROUP BY generated_date ORDER BY generated_date;
```

Знаходимо CAC та відношення

```
CAC = DIVIDE(SUM('public marketing_costs'[cost]), DISTINCTCOUNT('public
```



```
user_sub'[user_id]))
```

```
LTV =  
var profit_from_first_cohort = 75318 -- дохід січневої когорти мінус реальні  
витрати на рекламу за січень (SUM(marketing_costs.cost) за січень, без  
дублювання по користувачах)  
VAR users_from_first_cohort =  
    CALCULATE(          COUNT('public users'[user_id]),          MONTH('public  
users'[registration_date]) = 1)RETURN profit_from_first_cohort /  
users_from_first_cohort
```

```
LTV_CAC_RATIO = [LTV] / [CAC]
```

Після фіксу CAC по всій базі ~17, LTV січневої когорти ~19, LTV/CAC ~1.1 - економіка на межі окупності, а не в глибокому мініусі, як показувала версія з баганим costom.

Інші метрики , що вважаю умісними (KPI)

```
DAU = COUNT('public clickstream_logs'[user_id]) / DISTINCTCOUNT('public  
clickstream_logs'[Date])
```

```
MAU = COUNT('public clickstream_logs'[user_id]) / DISTINCTCOUNT('public  
clickstream_logs'[Month])
```

```
ARPU = SUM('public user_sub'[price_paid]) / COUNT('public  
user_sub'[user_id])
```

```
ARPPU = SUM('public user_sub'[price_paid]) / COUNT('public  
user_sub'[plan_id])
```

Фільтри :

- дата
- платформа
- план

Відслідковування економіки

price paid - чи купують підписку

dau - чи куплять її взагалі, та чи не поклали у нас сервер

Нижче візуалізація

